

Minimal Docker Commands to Demonstrate All Runnable Project Capabilities

Kiarash Rahmani

Run the commands in order from Windows Git Bash.

No.	Capability / Command Name	Command	What It Demonstrates
1	Enter project root	<code>cd "C:/Users/Kiarash Rahmani/Desktop/Codes/TMU/Sem2/LLM/Project/sentiment-analysis-mlops"</code>	Places the shell in the repository root so all relative paths and Compose commands work.
2	Validate Docker Compose	<code>docker compose config --quiet</code>	Verifies that the Compose file, services, ports, volumes, and environment definitions are syntactically valid.
3	Build project images	<code>docker compose build mlflow api app</code>	Builds the MLflow, FastAPI/ONNX, and training/utility images used by the project. This can be skipped when unchanged images already exist.
4	Start the complete stack	<code>docker compose up -d mlflow api</code>	Starts the MLflow tracking server and Docker FastAPI service. The API waits for MLflow, seeds a model when necessary, and loads the Production model.
5	Verify container health	<code>docker compose ps</code>	Confirms that both <code>sentiment-mlflow</code> and <code>sentiment-api</code> are running and healthy. Repeat after waiting if health is still starting.
6	Inspect API startup	<code>docker compose logs api --tail 50</code>	Shows MLflow connectivity, model seeding/loading, ONNX quantization attempts, micro-batching setup, and Uvicorn startup.
7	Health and optimization check	<code>curl -s http://localhost:8081/health</code>	Proves that the model is loaded and reports enabled serving optimizations such as vectorized batching, async micro-batching, and optional INT8 quantization.

Continued on next page

No.	Capability / Command Name	Command	What It Demonstrates
8	Model Registry information	<code>curl -s http://localhost:8081/model/info</code>	Displays the registered model name, Production stage, model URI, classes, and enabled optimizations.
9	Test every inference variant	<code>for v in baseline batched async_batched quantized; do echo "=== \$v ==="; curl -s -X POST "http://localhost:8081/predict?variant=\$v" -H "Content-Type: application/json" -d '{"text":"This movie was excellent and beautifully made."}'; echo; done</code>	Demonstrates sequential sklearn inference, vectorized inference, asynchronous micro-batching, and INT8 ONNX inference. A 503 for quantized means optional conversion was unavailable while the API remains functional.
10	Explicit batch prediction	<code>curl -s -X POST "http://localhost:8081/predict/batch?variant=batched" -H "Content-Type: application/json" -d '{"texts":["I loved this film","Terrible and boring movie","It was average"]}'</code>	Sends several reviews in one request and demonstrates the dedicated vectorized batch endpoint.
11	Pydantic validation	<code>curl -s -X POST http://localhost:8081/predict -H "Content-Type: application/json" -d '{"text":""}'</code>	Demonstrates request-schema validation by returning HTTP 422 for blank text.
12	Open MLflow UI	<code>cmd.exe /c start "" "http://localhost:5000"</code>	Opens the experiment-tracking and Model Registry interface containing runs, parameters, metrics, artifacts, and registered versions.
13	Open Swagger UI	<code>cmd.exe /c start "" "http://localhost:8081/docs"</code>	Opens the automatically generated FastAPI/OpenAPI documentation and interactive request interface.
14	Prefect data pipeline	<code>docker compose run --rm app -m src.data.ingest --input-path data/raw/imdb_train.csv --output-dir data/processed</code>	Runs the implemented Prefect ingestion flow: CSV loading, text cleaning, PII anonymization, stratified splitting, and processed-file generation.
15	Verify PII anonymization	<code>docker compose run --rm app -c "from src.data.processor import BasicTextCleaner; c=BasicTextCleaner(lowercase=True, remove_special=True, anonymize_pii=True); print(c.clean('Contact test@example.com or @john'))"</code>	Directly demonstrates masking of email addresses and usernames by the project text cleaner.

Continued on next page

No.	Capability / Command Name	Command	What It Demonstrates
16	Fast MLflow-tracked training	<code>docker compose run --rm app train.py --model-type baseline --sample-size 3000</code>	Trains TF-IDF plus Logistic Regression, evaluates it, saves the model and metrics, logs the run to MLflow, and registers a new model version.
17	Promote latest model	<code>docker compose run --rm app scripts/promote_model.py --stage Production</code>	Promotes the newest registered SentimentBaselineModel version to the MLflow Production stage.
18	Reload Production model	<code>docker compose restart api</code>	Restarts the serving container so it reloads the newly promoted Production model from MLflow.
19	Integration smoke tests	<code>docker compose run --rm app scripts/smoke_tests.py</code>	Tests MLflow connectivity, processed data, PII cleaning, model loading, prediction, evaluation, and experiment-run availability.
20	Inference benchmark	<code>docker compose exec api python scripts/run_benchmark.py --samples 100 --warmup 5 --runs 3 --output-dir reports/week3</code>	Measures baseline, batched, and optional quantized latency, throughput, memory, and model size; writes JSON and Markdown reports.
21	PSI drift detection	<code>docker compose exec api python -m src.monitoring.drift_detector --reference-path /app/data/processed/train.csv --production-path /app/logs/inference_data.csv</code>	Compares reference and production text-feature distributions using Population Stability Index.
22	Monitoring and alert reports	<code>cat reports/week4/monitoring_report.json reports/week4/alerts.jsonl</code>	Displays the existing monitoring health summary, drift/evaluation information, retraining status, and generated alerts.
23	Inspect generated artifacts	<code>docker compose exec api ls -lh /app/models /app/models/quantized /app/reports/week3 /app/logs</code>	Shows trained model files, optional ONNX/INT8 artifacts, benchmark reports, and serving logs mounted into the container.

Continued on next page

No.	Capability / Command Name	Command	What It Demonstrates
24	Safe shutdown	<code>docker compose down</code>	Stops and removes project containers and the Compose network while preserving the MLflow named volume, models, data, logs, and reports.